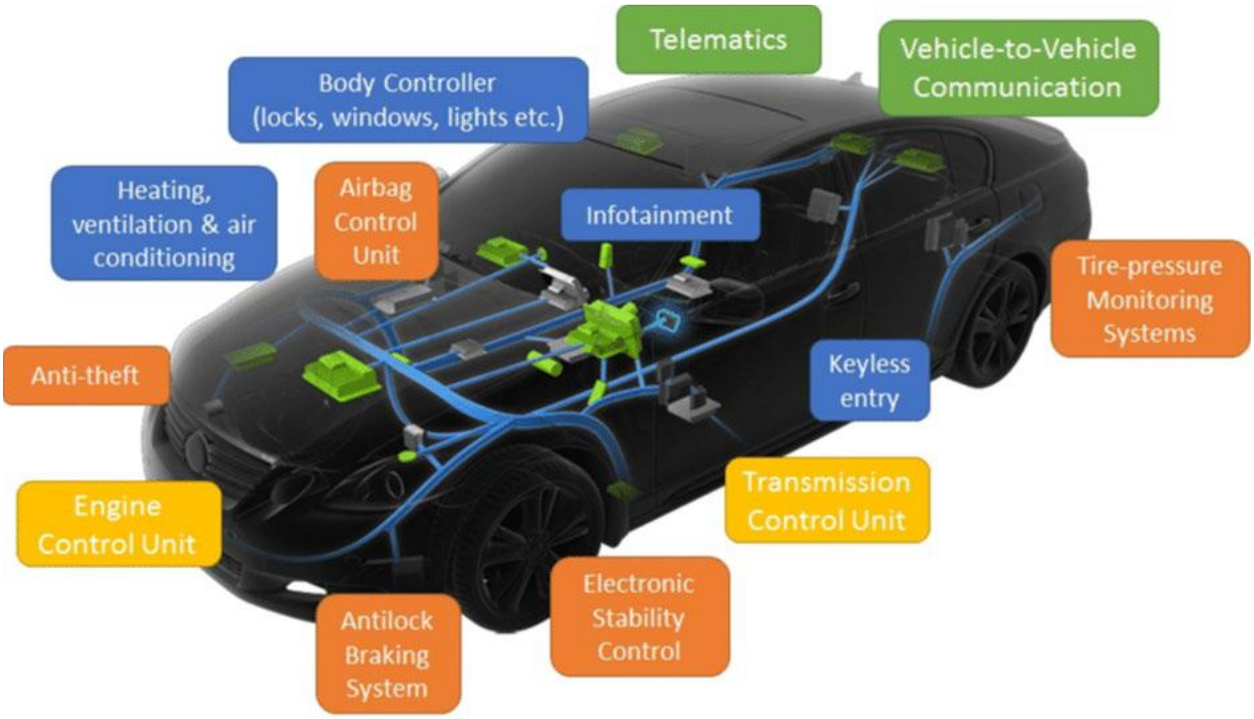
Car Simulation Project

Design Document

Sem3 Autumn 2024



Plamen Tsokov & Jovan Christopher

Introduction

The project will consist of 4 ECUs which are connected through a CAN bus, each is responsible for the lights, engine, tires, and a communication ECU which receives data from the other ECUs and passes data to a dashboard and a remote-control app.

Communication with the remote-control app will be done wirelessly, so the ESP32 MCU was chosen for this project. To generalize the code and platform.io project, all 4 ECUs are going to be ESP32s.

System Description

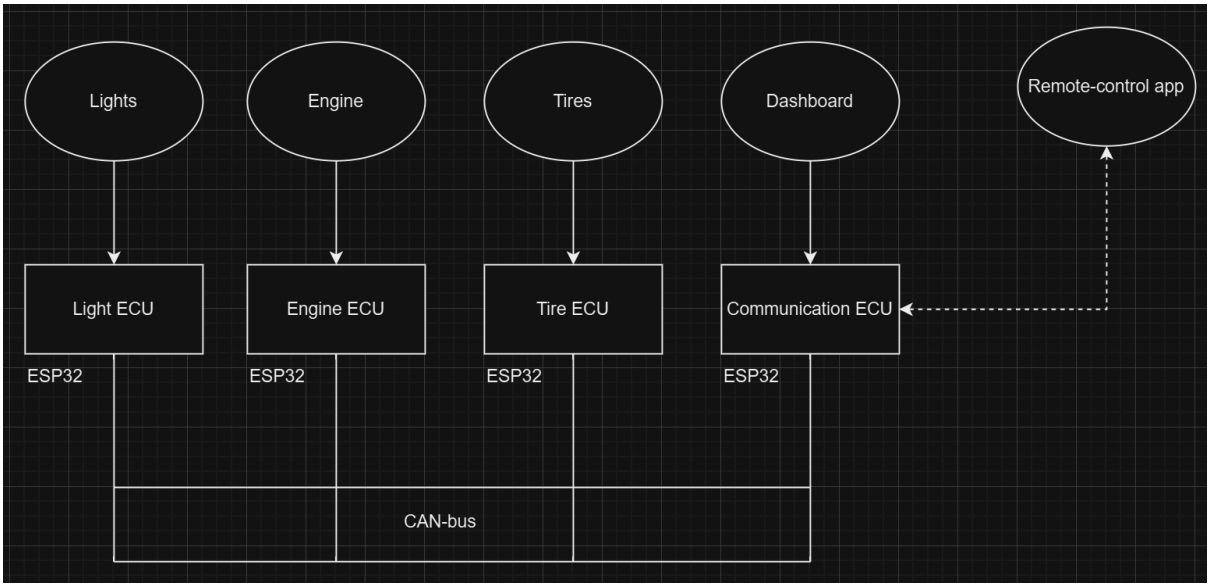
Environment Interaction

Only the communication ECU has access to the outside world where it can send and receive data.

The remote-control app will be Node-Red dashboard that could have buttons and inject values. The communication protocol that will be used is going to be MQTT. MQTT protocol is reliable to guarantee that the subscriber will receive the message at least once, it uses a low bandwidth making the messages easier to transmit and thus ensuring the communication stays consistent. The subscriber can virtually access the published data from anywhere if they have internet connection.

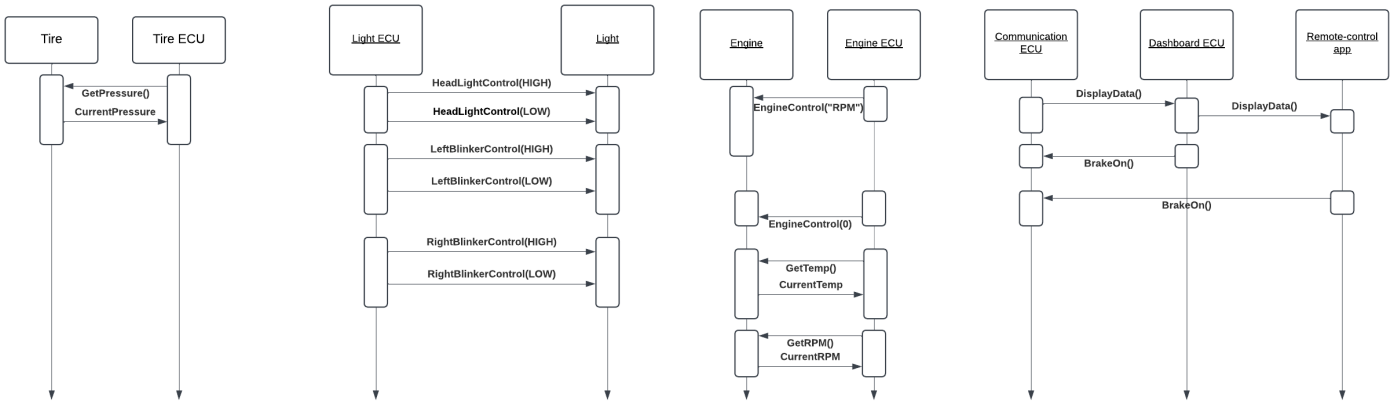
These advantages could demonstrate the car simulation to be very close to a real world implementation.

Structure

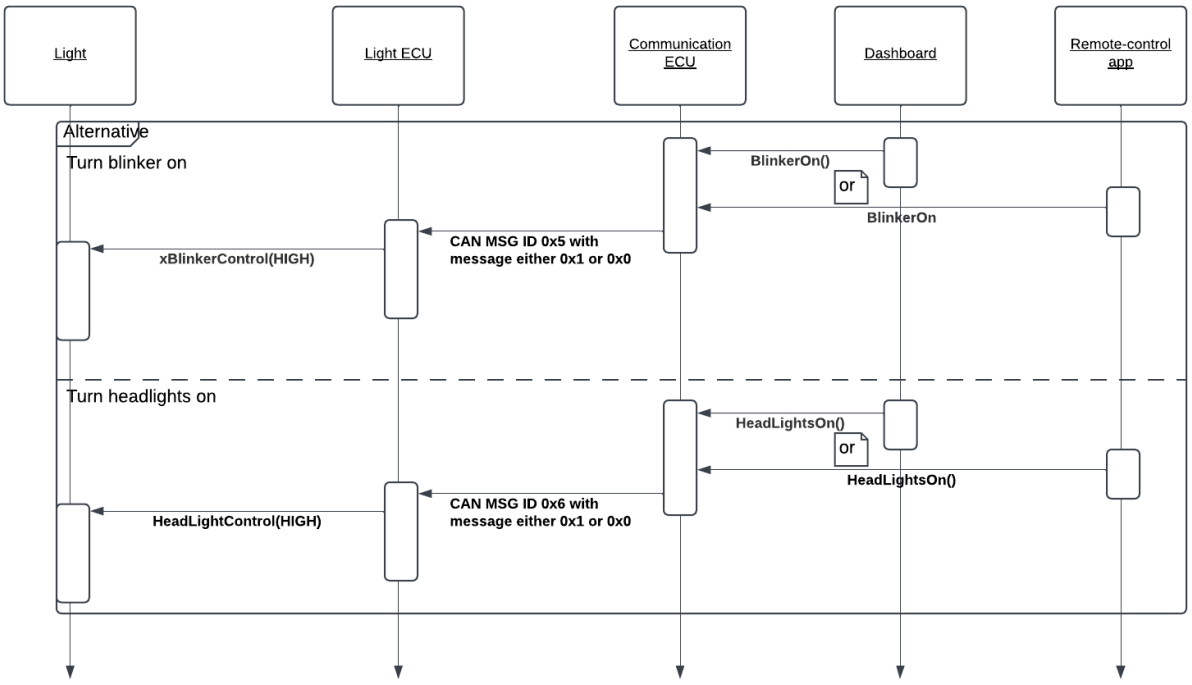


ECUs will be connected with each other through a CAN bus and the Communication ECU sends data received from the other ECUs to a dashboard and to a remote-control app.

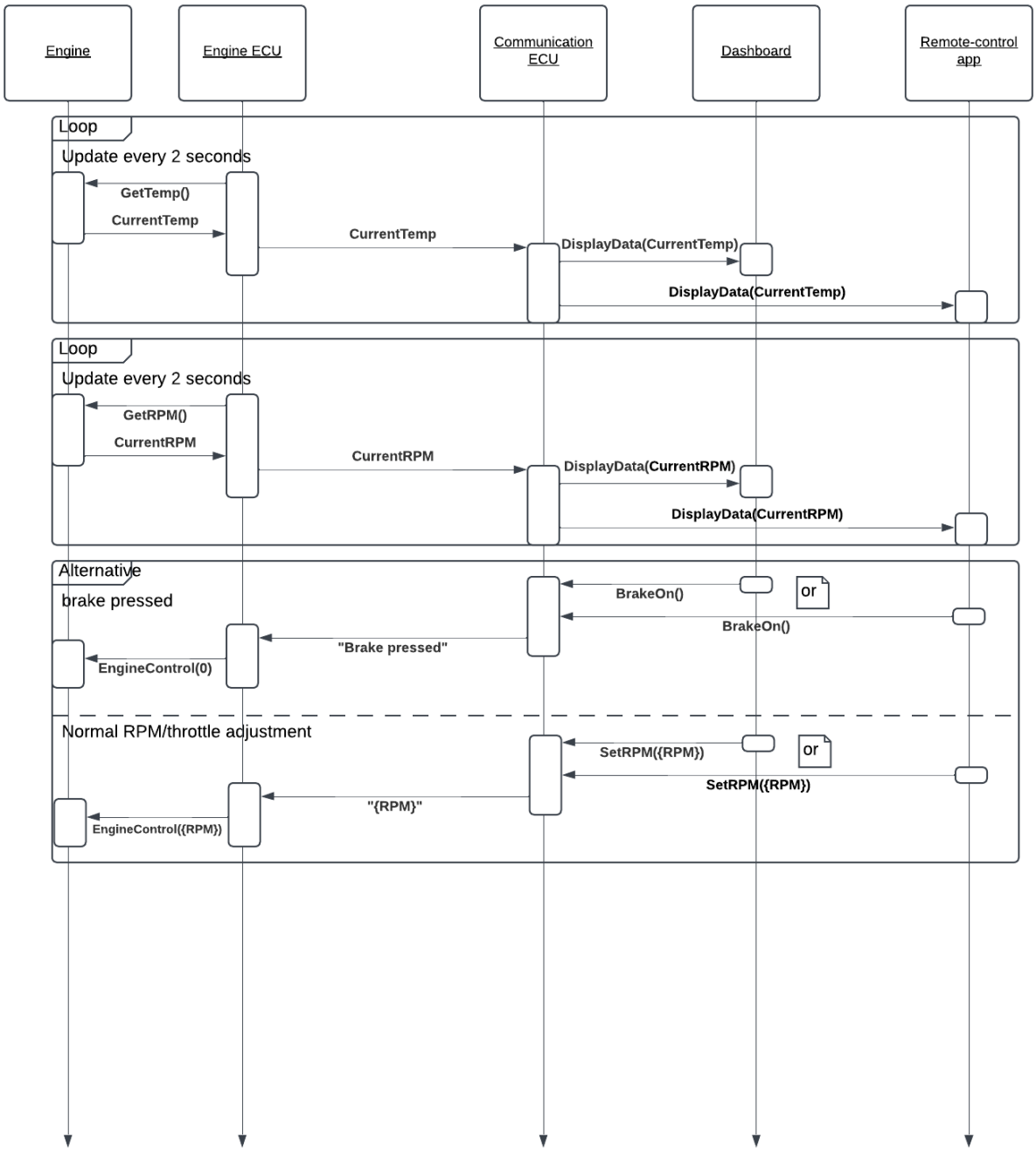
Sequence Diagram



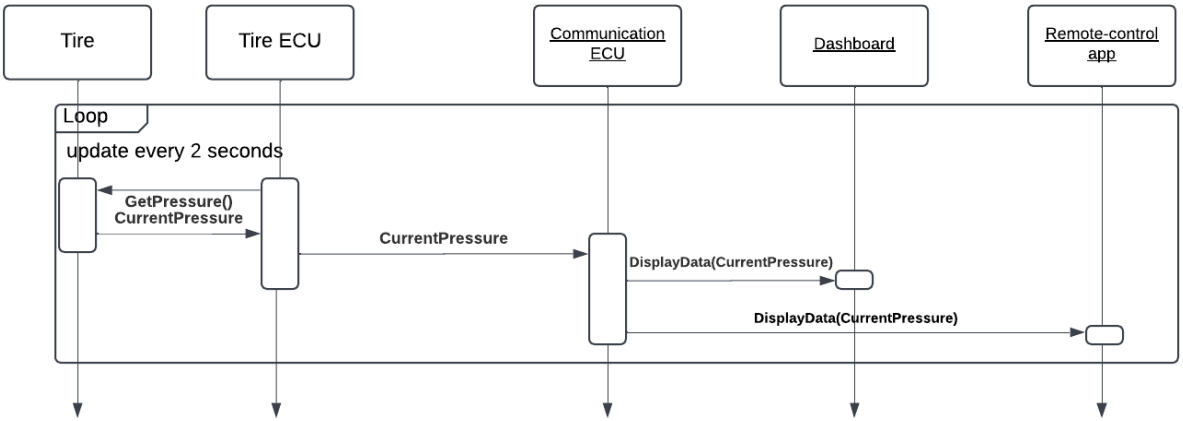
Interaction between class objects within each ECUs.



Interaction between Light ECU and Communication ECU



Interaction between Engine ECU and the Communication ECU to display data.

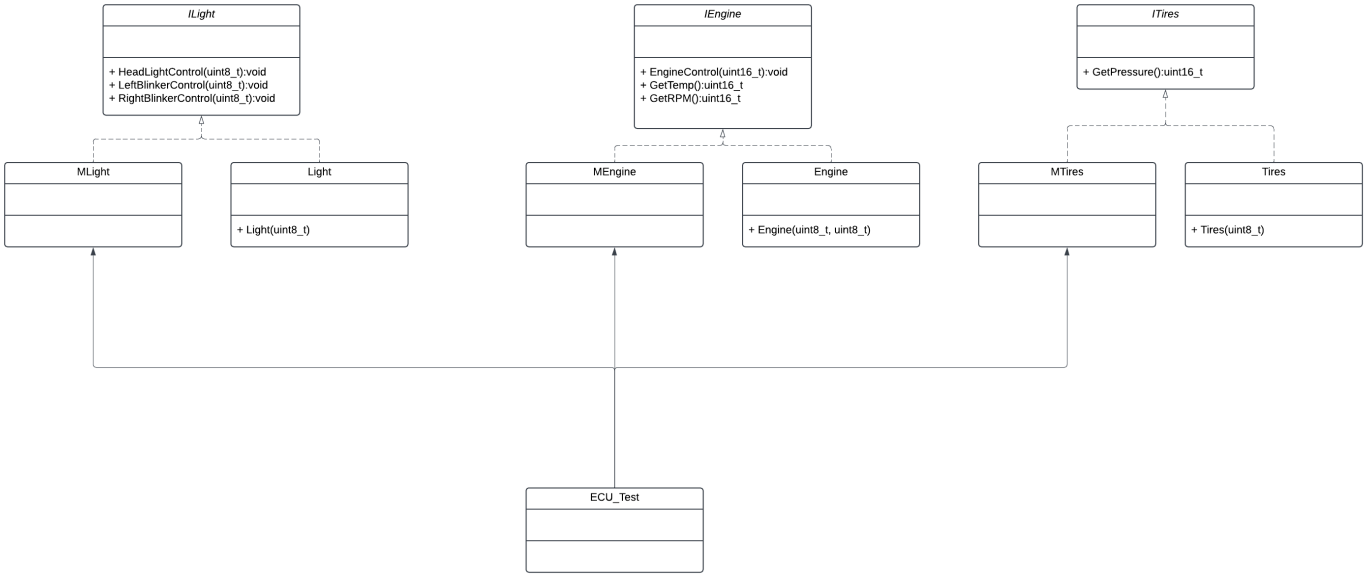


Interaction between Tire ECU and Communication ECU

Message protocols

ID	Message	Interval	Description
0x1	“Brake pressed”		Highest priority
0x2	“{ENGINETEMP}°C”	2 seconds	
0x3	“{TIREPRESSURE}PSI”	2 seconds	
0x4	“{RPM}”		Changes RPM of the engine
0x5	“0x1” or “0x0”		Sends a high or low bit to control blinkers
0x6	“0x1” or “0x0”		Sends a high or low bit to control headlights

Class Diagram



The test program will make use of mock classes. What is not shown in the diagram is each ECUs will use its own corresponding classes. So each ECUs will use 1 of the interfaces depending on its responsibility.

Testing Strategy

Communication ECU will have a mock test using google mock framework to test incoming data and its behavior towards those data. The other ECUs have proof of concept and unit tests if applicable.

Chosen Hardware

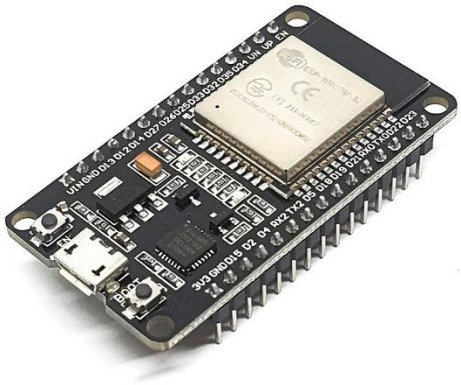


Image courtesy of:
https://www.bing.com/aclick?ld=e8hMegj9Tp4221f4d7VLmHZDVUCUwqXakKueI2ebc7PntiXxAtVHKVwopVqOI8QYZyYNIkOkyiovIeasxchMem92podPjoE6Tbbb2hxz24fO6zDKQDWCsbrSurocbP7W80G2w95dzWgiBaK3koOrgJ-fbi2kBSTRAc9tlMC2TroQyV3I_f&u=aHR0cHMlM2ElMmYlMmZ3d3cuYW1hem9uLm5sJTJmcyUyZiUzZmllJTNkVVRGOCUyNmtleXdvcmRzJTNkZXNwMzIlMmJkZXZlbG9wbWVudCUyYmJvYXJkJTI2aW5kZXglM2RhcHMlMjZ0YWclM2RubHR4dGJpc3RkZGUtMjElMjZyZWYlM2RwZF9zbF82amtleXljMXE3X3AlMjZhZGdycGlkJTNkMTMzNTkwNzA5NjQwMDAzNyUyNmh2YWRpZCUzZDgzNDk0MjU1MjE1Nzg5JTI2aHZuZXR3JTNkbyUyNmh2cW10JTNkcCUyNmh2Ym10JTNkYnAlMjZodmRldiUzZGMlMjZodmxvY2ludCUzZCUyNmh2bG9jcGh5JTNkMTU5OTE3JTI2aHZ0YXJnaWQlM2Rrd2QtODM0OTQ1OTAwMzc3MjklM2Fsb2MtMTI5JTI2aHlkYWRjciUzZDE2NTY4XzE5NjQ3MTYlMjZtc2Nsa2lkJTNkZDYzZGRmMzU3MTRjMTAwYTQ1ZjhiODdhNzc5MGM3Y2Y&rlid=d63ddf35714c100a45f8b87a7790c7cf&ntb=1

ESP32 was chosen because there needs to be a connection to a remote-control app which is Node-Red

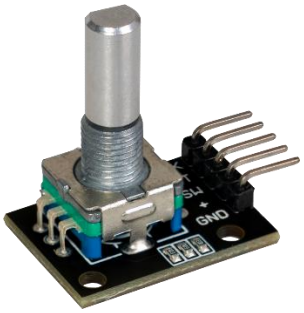


Image courtesy of:
https://www.bing.com/aclick?ld=e8yANH5xxOD9SM_lAg94uHijVUCUxms9aFPugQWDXGVBnvGd7v3dF4MWt7xgXH9rXcf7m8OF3XjfAosezCWnxU9nz2Ac2mjCsGQ69-NfwmDD2eDQR9X4BOuFzQ38wY0DGUnwHOZ7bVftkMUJ-I77zz1BBiFwuJba8w5ZT_CEbYac3XBUYv&u=aHR0cHMlM2ElMmYlMmZ3d3cuYW1hem9uLm5sJTJmcyUyZiUzZmllJTNkVVRGOCUyNmtleXdvcmRzJTNka3ktMDQwJTJicm90YXJ5JTJiZW5jb2RlciUyNmluZGV4JTNkYXBzJTI2dGFnJTNkbmx0eHRiaXN0ZGRlLTIxJTI2cmVmJTNkcGRfc2xfNHprenJwdW4yNF9lJTI2YWRncnBpZCUzZDEzMzkyMDY3ODY3NDQ4NDAlMjZodmFkaWQlM2Q4MzcwMDczNTQyMTc4OCUyNmh2bmV0dyUzZG8lMjZodnFtdCUzZGJlJTI2aHZkZXYlM2RjJTI2aHZsb2NpbnQlM2QlMjZodmxvY3BoeSUzZDE1OTkxNyUyNmh2dGFyZ2lkJTNka3dkLTgzNzAxNDMwMDQ0NDQzJTNhbG9jLTEyOSUyNmh5ZGFkY3IlM2QxNjU3MF8yMjU3NDc1JTI2bXNjbGtpZCUzZGQyZGFmYzM5NDQ2NTFlNjk0OWExOWI3NjQzMGUxZTI2&rlid=d2dafc3944651e6949a19b76430e1e26&ntb=1

The KY-040 potentiometer will be used to simulate the tire pressures.



Image courtesy of: https://core-electronics.com.au/parallax-feedback-360-high-speed-servo.html

The parallax 360 servo will be used to simulate the engine.



Image courtesy of: https://anlandpremium.vn/wvvhn-69359-Module-Can-Bus-Module-Shield-V-DC/

MCP2515 CAN module which uses SPI to connect to an MCU, used as CAN communication interface.